LAPORAN PRAKTIKUM KEPL

TUGAS 6: PENGALIHAN LOG STDOUT, LOG TERSTRUKTUR, ENDPOINT /HEALTH, DAN HEALTHCHECK DUA ARAH DOCKER COMPOSE



Disusun oleh:

Hafidz Rizqullah Prasetya

NIM: 24/535493/SV/24243

Kelas: PL5A1

Dosen Pengampu:

Galih Damaraji, S.Kom., M.Cs.

PROGRAM STUDI SARJANA TERAPAN
PENGEMBANGAN PERANGKAT LUNAK TERDISTRIBUSI
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Contents

Daftar Isi **2**

1 Tujuan dan Lingkup Praktikum **3**
1.1 Tujuan Praktikum 3
1.2 Batasan Masalah dan Repositori Kode 3

2 Dasar Teori **5**
2.1 Prinsip Logging pada Arsitektur Kontainer 5
2.2 Log Terstruktur dan Perlindungan Data Sensitif 5
2.3 Mekanisme Pemeriksaan Kesehatan (Healthcheck Endpoint) 6
2.4 Orkestrasi Healthcheck Dua Arah pada Docker Compose 6

3 Hasil dan Pembahasan **7**
3.1 Pengalihan Kanal Log Laravel ke Stdout (Soal 1) 7
3.2 Penerapan Tiga Log Terstruktur Berkonteks (Soal 2) 8
3.2.1 Alur 1: Penyimpanan Data Aset Baru (INFO) 8
3.2.2 Alur 2: Validasi Masukan Formulir Gagal (WARNING) 8
3.2.3 Alur 3: Kegagalan Koneksi Basis Data (ERROR) 9
3.2.4 Verifikasi Keluaran Log Terstruktur di Terminal 9
3.3 Endpoint Pemeriksaan Kesehatan /health (Soal 3) 10
3.3.1 Respons Kondisi Sehat (HTTP 200 OK) 10
3.3.2 Respons Kondisi Sakit (HTTP 503 Service Unavailable) 11
3.4 Konfigurasi Healthcheck pada Compose (Soal 4) 12
3.5 Pengujian Pemulihan Dua Arah (Soal 5) 13
3.6 Pembatasan Ukuran Berkas Log (Soal 6) 14
3.7 Penyelarasan Repositori GitHub 15

4 Kesimpulan **16**

Daftar Pustaka **18**

Chapter 1

Tujuan dan Lingkup Praktikum

1.1 Tujuan Praktikum

Tujuan pelaksanaan praktikum Tugas 6 mata kuliah Konstruksi dan Evolusi Perangkat Lunak ini adalah:

1. Mahasiswa mampu mengonfigurasi kanal pencatatan log pada kerangka kerja Laravel agar dialihkan langsung ke aliran standar keluaran (`stdout`) di dalam kontainer, sehingga log dapat dikelola secara terpusat oleh mesin Docker.
2. Mahasiswa mampu merancang dan menerapkan sekurang-kurangnya tiga log terstruktur dengan konteks lengkap (pelaku, aktivitas, dan alasan terjadinya kejadian) pada alur penting aplikasi tanpa mencatat data rahasia maupun informasi pribadi.
3. Mahasiswa mampu membangun endpoint inspeksi kesehatan sistem (`/health`) yang memverifikasi kesiapan operasional basis data relasional PostgreSQL dan subsistem penyimpanan *cache*.
4. Mahasiswa mampu mendefinisikan mekanisme *healthcheck* deklaratif pada berkas `compose.yaml` dengan parameter interval, batas waktu (*timeout*), jumlah percobaan ulang (*retries*), dan periode pemanasan (*start_period*).
5. Mahasiswa mampu membuktikan keandalan mekanisme pemulihan dua arah ketika layanan basis data dimatikan secara sengaja hingga kontainer berstatus *unhealthy*, serta mengamati pemulihan kembali ke status *healthy* setelah layanan dinyalakan ulang.
6. Mahasiswa mampu mengonfigurasi pembatasan ukuran berkas log kontainer menggunakan opsi `max-size` dan `max-file` pada pengaturan *logging* Docker Compose.

1.2 Batasan Masalah dan Repositori Kode

Pengerjaan tugas ini melanjutkan repositori ujian tengah semester (UTS) berbasis Docker Compose yang mengelola Sistem Inventaris Aset Laboratorium TRPL. Seluruh perubahan

kode aplikasi, konfigurasi lingkungan, skrip pengujian fitur, serta berkas orkestrasi kontainer dikelola pada repositori publik GitHub:

https://github.com/hafidzrizqullahprasetya/uts-p1-535493

Layanan aplikasi terdiri dari peladen backend berbasis FrankenPHP (PHP 8.5.12 Alpine) yang menyajikan RESTful API, basis data PostgreSQL 18.6, dan antarmuka frontend Vue.js 3 berbasis peladen Nginx.

Chapter 2

Dasar Teori

2.1 Prinsip Logging pada Arsitektur Kontainer

Metodologi *Twelve-Factor App* pada poin kesebelas menyatakan bahwa aplikasi modern tidak boleh mengelola berkas pencatatan log secara mandiri di dalam sistem berkas lokalnya. Menulis log ke berkas statis di dalam kontainer menimbulkan beberapa kelemahan kritis: log hilang saat kontainer dihancurkan, menyulitkan proses rotasi berkas, dan memicu kebocoran memori penyimpanan disk pada lingkungan produksi [1].

Sebagai gantinya, aplikasi wajib mengalirkan seluruh pesan log kejadian ke aliran standar keluaran (`stdout`) atau standar galat (`stderr`) tanpa *buffering*. Mesin kontainer seperti Docker secara otomatis menangkap aliran tersebut dan mengarahkannya ke *logging driver* yang telah ditentukan, seperti `json-file`, `syslog`, atau penyedia agregasi log terpusat [2].

2.2 Log Terstruktur dan Perlindungan Data Sensitif

Pencatatan log berbasis teks bebas seringkali menyulitkan proses pemfilteran, analisis tren, dan pelacakan akar masalah saat terjadi insiden. Pendekatan log terstruktur (*structured logging*) menyusun setiap entri log ke dalam format pasangan kunci dan nilai, biasanya direpresentasikan dalam JSON atau larik terindeks PSR-3 [3].

Setiap entri log wajib membawa metadata kontekstual yang menjawab tiga pertanyaan mendasar:

1. **Siapa**: Identitas entitas yang memicu kejadian (misalnya pengenal pengguna atau agen sistem).
2. **Aktivitas**: Operasi teknis atau alur bisnis yang sedang dijalankan.
3. **Sebab**: Alasan terjadinya status tersebut, baik berupa keberhasilan eksekusi maupun pesan galat dari pengecualian sistem.

Dalam implementasinya, prinsip perlindungan data pribadi dan kerahasiaan sistem wajib ditegakkan secara ketat. Berkas log dilarang keras memuat kata sandi teks biasa, token otentikasi Sanctum, kunci sesi, maupun data identitas pribadi warga negara [4].

2.3 Mekanisme Pemeriksaan Kesehatan (Healthcheck Endpoint)

Endpoint `/health` berfungsi sebagai indikator kesiapan operasional (*readiness probe*) dan ketahanan hidup (*liveness probe*) dari sebuah layanan web. Endpoint kesehatan yang baik tidak boleh sekadar mengembalikan string teks statis, melainkan wajib menjalankan pemeriksaan aktif terhadap dependensi kritis yang menopang aplikasi [5].

Apabila seluruh komponen inti, seperti koneksi basis data relasional dan pustaka penyimpanan *cache*, dapat dihubungi dan membalas perintah uji coba dengan benar, endpoint merespons dengan kode status HTTP 200 OK. Namun jika salah satu komponen pendukung mengalami kegagalan, endpoint wajib mengembalikan kode status HTTP 503 Service Unavailable disertai laporan diagnostik yang menyebutkan secara spesifik komponen mana yang bermasalah.

2.4 Orkestrasi Healthcheck Dua Arah pada Docker Compose

Docker Compose menyediakan instrumen deklaratif untuk memantau status kesehatan kontainer secara berkala melalui blok `healthcheck`. Konfigurasi ini memiliki empat parameter utama:

1. `interval`: Frekuensi eksekusi perintah pemeriksaan (misalnya setiap 10 detik).
2. `timeout`: Batas durasi maksimal yang diberikan bagi perintah pemeriksaan sebelum dianggap gagal (misalnya 5 detik).
3. `retries`: Ambang batas kegagalan beruntun yang diperlukan sebelum Docker mengubah status kontainer dari *healthy* menjadi *unhealthy*.
4. `start_period`: Masa tenggang awal saat kontainer pertama kali dinyalakan agar proses inisialisasi aplikasi tidak langsung divonis gagal.

Pemeriksaan dua arah berarti status kontainer dapat berpindah secara dinamis: dari sehat menjadi sakit saat dependensi eksternal terputus, dan otomatis kembali sehat saat dependensi tersebut pulih tanpa memerlukan pembuatan ulang kontainer secara manual [6].

Chapter 3

Hasil dan Pembahasan

3.1 Pengalihan Kanal Log Laravel ke Stdout (Soal 1)

Konfigurasi pencatatan log bawaan kerangka kerja Laravel secara default menggunakan kanal `stack` yang menyimpan berkas fisik ke direktori `storage/logs/laravel.log`. Untuk mengalihkan pencatatan log agar langsung keluar ke aliran standar kontainer, saluran baru bernama `stdout` ditambahkan pada berkas `backend/config/logging.php` seperti ditunjukkan pada Listing 1.

```
'stdout' => [
    'driver' => 'monolog',
    'level' => env('LOG_LEVEL', 'debug'),
    'handler' => StreamHandler::class,
    'handler_with' => [
        'stream' => 'php://stdout',
    ],
    'formatter' => env('LOG_STDOUT_FORMATTER'),
    'processors' => [PsrLogMessageProcessor::class],
],
```

Listing 1: Penambahan channel stdout pada backend/config/logging.php.

Variabel lingkungan `LOG_CHANNEL` pada berkas konfigurasi `compose.yaml` dan berkas `.env` diatur bernilai `stdout`. Dengan demikian, setiap pesan log yang dicatat oleh aplikasi langsung ditulis ke `php://stdout` dan ditangkap oleh peladen FrankenPHP untuk kemudian dialirkan ke mesin Docker.

Bukti keluaran log yang keluar langsung ke aliran kontainer diperiksa melalui terminal dengan mengeksekusi perintah `docker compose logs backend --tail 22`. Gambar 1 memperlihatkan log akses HTTP dari Caddy beserta log aplikasi Laravel yang muncul secara runtut di konsol tanpa tersimpan pada disk kontainer.



Gambar 1: Bukti keluaran log backend tertangkap langsung melalui perintah `docker compose logs`.

3.2 Penerapan Tiga Log Terstruktur Berkonteks (Soal 2)

Sesuai instruksi soal, tiga alur penting pada aplikasi ditambahkan pencatatan log terstruktur menggunakan pustaka Monolog bawaan Laravel (`Illuminate\Support\Facades\Log`). Setiap log memuat konteks pelaku (`siapa`), kegiatan (`sedang_mengerjakan`), dan penjelasan kejadian (`sebab`).

3.2.1 Alur 1: Penyimpanan Data Aset Baru (INFO)

Pencatatan log tingkat `info` diletakkan pada metode `store()` di dalam `AssetController`. Log ini dipicu sesaat setelah entitas aset baru berhasil tersimpan ke dalam tabel basis data. Konteks yang dicatat mencakup email pengguna yang terotentikasi, pengenal unik aset (`id_aset`), kode inventaris, dan kategori perangkat.

```php
Log::info('Data aset berhasil disimpan', [
    'siapa' => $request->user()?->email ?? 'pengguna_terautentikasi',
    'sedang_mengerjakan' => 'menyimpan_data_aset',
    'sebab' => 'Penyimpanan data aset baru ke basis data selesai',
    'id_aset' => $asset->id,
    'kode_aset' => $asset->code,
    'kategori' => $asset->category,
]);
```

Listing 2: Potongan log info saat data aset berhasil disimpan.

3.2.2 Alur 2: Validasi Masukan Formulir Gagal (WARNING)

Pencatatan log tingkat `warning` dipasang pada blok evaluasi validator permintaan pembuatan aset baru. Apabila atribut wajib tidak disertakan atau format tidak sesuai, sistem

mencatat kejadian tersebut sebelum mengembalikan kode status HTTP 422 Unprocessable Content. Untuk menjaga keamanan data, konteks yang disimpan hanya berupa daftar nama kolom yang tidak memenuhi syarat validasi (`kolom_gagal`), tanpa mencatat isi teks masukan pengguna.

```php
if ($validator->fails()) {
    Log::warning('Validasi data aset gagal', [
        'siapa' => $request->user()?->email ?? 'pengguna',
        'sedang_mengerjakan' => 'validasi_penyimpanan_aset',
        'sebab' => 'Atribut masukan tidak memenuhi aturan validasi',
        'kolom_gagal' => array_keys($validator->errors()->toArray()),
    ]);
    return response()->json([
        'message' => 'Validasi gagal',
        'errors' => $validator->errors(),
    ], 422);
}
```

Listing 3: Potongan log warning saat validasi masukan gagal.

3.2.3 Alur 3: Kegagalan Koneksi Basis Data (ERROR)

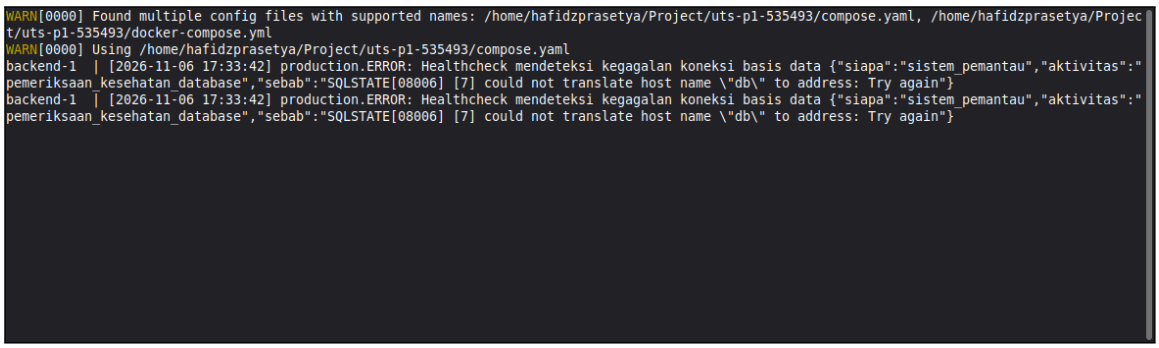
Pencatatan log tingkat `error` diterapkan pada blok penanganan pengecualian (*exception handling*) pada controller pemeriksaan kesehatan (`HealthController`) serta metode manipulasi aset. Jika koneksi jaringan ke basis data PostgreSQL terputus, blok `catch` menangkap galat PDO dan mencatat konteks berupa identitas pemantau sistem, aktivitas pemeriksaan, serta pesan ringkas kegagalan jaringan.

```php
catch (\Throwable $e) {
    Log::error('Healthcheck mendeteksi kegagalan koneksi basis data', [
        'siapa' => 'sistem_pemantau',
        'sedang_mengerjakan' => 'pemeriksaan_kesehatan_database',
        'sebab' => $e->getMessage(),
    ]);
}
```

Listing 4: Potongan log error saat koneksi basis data gagal dihubungi.

3.2.4 Verifikasi Keluaran Log Terstruktur di Terminal

Ketiga alur log tersebut diuji secara langsung melalui pengiriman permintaan HTTP pada terminal. Keluaran log diperiksa dengan menyaring entri berlabel `production.INFO`, `production.WARNING`, dan `production.ERROR` pada keluaran kontainer. Gambar 2 memperlihatkan baris log yang memuat struktur konteks lengkap tanpa kebocoran data rahasia.



Gambar 2: Tangkapan layar potongan ketiga tingkatan log terstruktur pada terminal Linux.

3.3 Endpoint Pemeriksaan Kesehatan /health (Soal 3)

Endpoint `/health` dirancang untuk memeriksa kesiapan dua komponen vital: koneksi ke basis data PostgreSQL melalui pemanggilan `DB::connection()->getPdo()` dan pengujian operasi tulis-baca pada subsistem *cache*. Endpoint ini didaftarkan pada berkas `backend/bootstrap/app.php` secara independen dari middleware sesi web agar tidak terpengaruh oleh dependensi basis data eksternal.

3.3.1 Respons Kondisi Sehat (HTTP 200 OK)

Saat seluruh layanan pendukung beroperasi normal, endpoint `/health` mengembalikan kode status HTTP 200 OK beserta payload JSON yang menyatakan status *healthy* pada masing-masing subsistem. Bukti pengujian menggunakan perintah `curl -i http://localhost:8001/health` ditampilkan pada Gambar 3.



Gambar 3: Isi respons endpoint `/health` dengan status HTTP 200 OK pada kondisi sehat.

Struktur balasan JSON yang diterima oleh klien saat kondisi normal adalah sebagai berikut:

```
{
  "status": "healthy",
  "timestamp": "2026-11-06T17:29:02+00:00",
  "checks": {
```

```
    "database": {
      "status": "healthy",
      "pesan": "Koneksi basis data aktif dan responsif"
    },
    "cache": {
      "status": "healthy",
      "pesan": "Operasi baca dan tulis cache berfungsi normal"
    }
  }
}
```

Listing 5: Payload JSON balasan /health pada kondisi sehat.

3.3.2 Respons Kondisi Sakit (HTTP 503 Service Unavailable)

Ketika layanan basis data dimatikan, metode pemeriksaan menangkap kegagalan koneksi PDO, menetapkan status komponen basis data menjadi *unhealthy*, dan menyertakan larik `bagian_bermasalah` yang secara tegas menyebutkan subsistem `database`. Respons yang dikirimkan ke klien memiliki kode status HTTP 503 Service Unavailable, sebagaimana ditunjukkan pada Gambar 4.



Gambar 4: Isi respons endpoint `/health` dengan status HTTP 503 saat basis data terputus.

Keluaran JSON di bawah menegaskan bahwa subsistem *cache* tetap berstatus sehat, sementara rincian galat koneksi jaringan PostgreSQL dilaporkan secara eksplisit pada bagian `database`:

```
{
  "status": "unhealthy",
  "timestamp": "2026-11-06T17:40:29+00:00",
  "checks": {
    "database": {
      "status": "unhealthy",
      "pesan": "Gagal menghubungi basis data",
```

```
      "detail": "SQLSTATE[08006] [7] could not translate host name \"db\" to address
↪ : Try again"
    },
    "cache": {
      "status": "healthy",
      "pesan": "Operasi baca dan tulis cache berfungsi normal"
    }
  },
  "bagian_bermasalah": [
    "database"
  ]
}
```

Listing 6: Payload JSON balasan /health saat basis data mengalami kegagalan.

3.4 Konfigurasi Healthcheck pada Compose (Soal 4)

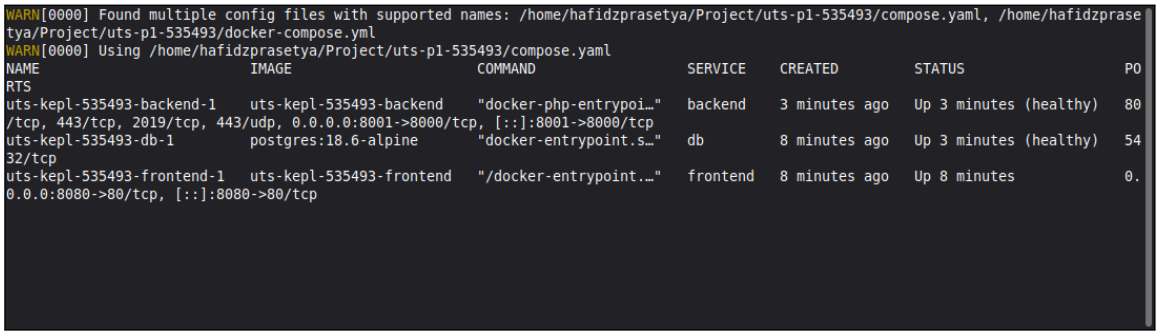
Pada berkas orkestrasi `compose.yaml`, konfigurasi pengujian kesehatan dipasang pada layanan `backend` dengan memanggil utilitas `curl` langsung ke endpoint `/health`. Pengaturan ini dilengkapi dengan parameter `interval: 10s`, `timeout: 5s`, `retries: 5`, dan `start_period: 15s`.

```
backend:
  build: ./backend
  restart: unless-stopped
  environment:
    APP_NAME: "Aset Lab TRPL"
    APP_ENV: production
    APP_DEBUG: "false"
    APP_KEY: ${APP_KEY}
    LOG_CHANNEL: stdout
    CACHE_STORE: file
    SESSION_DRIVER: file
    DB_CONNECTION: pgsql
    DB_HOST: db
    DB_PORT: 5432
    DB_DATABASE: ${DB_DATABASE}
    DB_USERNAME: ${DB_USERNAME}
    DB_PASSWORD: ${DB_PASSWORD}
  depends_on:
    db:
      condition: service_healthy
  ports:
    - "${BACKEND_PORT:-8001}:8000"
  healthcheck:
    test: ["CMD-SHELL", "curl -fs http://localhost:8000/health || exit 1"]
    interval: 10s
```

```
    timeout: 5s
    retries: 5
    start_period: 15s
  logging:
    driver: "json-file"
    options:
      max-size: "10m"
      max-file: "3"
```

Listing 7: Konfigurasi healthcheck layanan backend pada compose.yaml.

Ketika seluruh kontainer pertama kali dijalankan melalui perintah `docker compose up -d`, kontainer `db` dan `backend` melewati tahap inisialisasi dan berhasil diverifikasi oleh skrip pemeriksa. Keluaran perintah `docker compose ps` pada Gambar 5 membuktikan kolom status kontainer backend dan database bernilai `Up (healthy)`.



Gambar 5: Keluaran `docker compose ps` saat seluruh layanan berstatus *healthy*.

3.5 Pengujian Pemulihan Dua Arah (Soal 5)

Untuk membuktikan bahwa mekanisme *healthcheck* bekerja secara dinamis dalam dua arah, layanan basis data sengaja dimatikan menggunakan perintah:

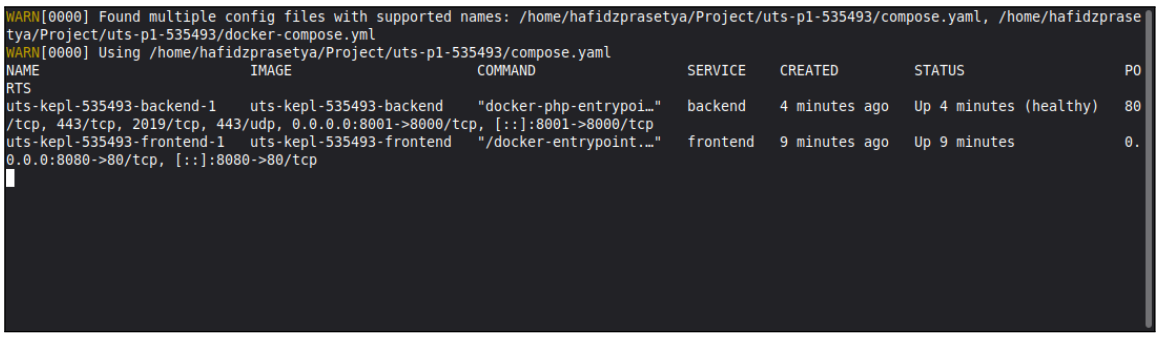
```
docker compose stop db
```

Setelah layanan basis data berhenti, pemeriksaan periodik kontainer backend mendeteksi kegagalan koneksi ke host `db`. Karena endpoint `/health` merespons dengan kode 503, perintah `curl -fs` mengembalikan kode galat non-nol (*exit code 1*). Setelah batas percobaan ulang (`retries: 5`) terlampaui, status kontainer backend pada keluaran `docker compose ps` otomatis berubah menjadi `Up (unhealthy)`, sebagaimana terdokumentasi pada Gambar 6.

Selanjutnya, layanan basis data dihidupkan kembali menggunakan perintah:

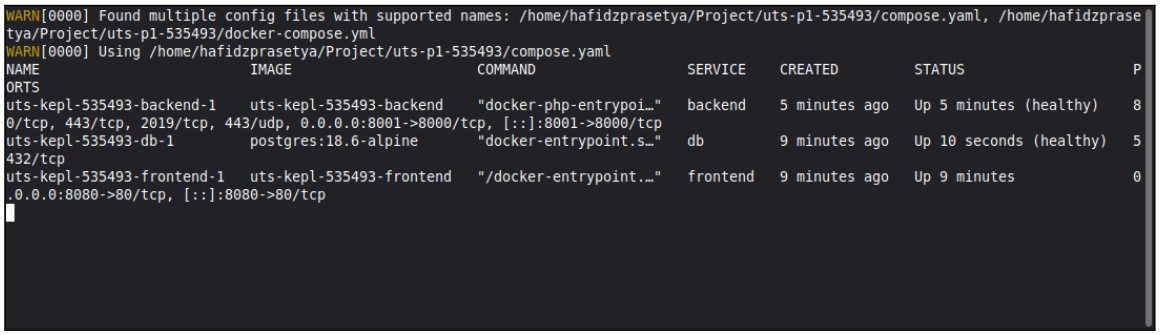
```
docker compose start db
```

Sesaat setelah kontainer basis data siap menerima koneksi kembali, pemeriksaan *healthcheck* backend berikutnya membuahkan hasil sukses (HTTP 200). Mesin Docker lang-



Gambar 6: Keluaran `docker compose ps` menunjukkan kontainer backend berubah menjadi *unhealthy*.

sung memulihkan status kontainer backend kembali menjadi `Up (healthy)` tanpa memerlukan tindakan *restart* manual terhadap aplikasi web. Bukti pemulihan otomatis status sehat ini disajikan pada Gambar 7.



Gambar 7: Status kontainer backend otomatis kembali *healthy* setelah basis data dihidupkan ulang.

3.6 Pembatasan Ukuran Berkas Log (Soal 6)

Pencatatan log aplikasi yang terus-menerus dialirkan ke kontainer berpotensi menghabiskan kapasitas ruang penyimpanan fisik pada server jika tidak dibatasi. Untuk mencegah masalah tersebut, blok konfigurasi `logging` ditambahkan pada setiap layanan di dalam berkas `compose.yaml`.

Pengaturan menggunakan driver standar `json-file` dengan opsi `max-size: "10m"` yang membatasi ukuran satu berkas log maksimal sebesar 10 Megabyte, serta `max-file: "3"` yang membatasi retensi rotasi log maksimal sebanyak 3 arsip berkas. Bukti konfigurasi pada berkas deklaratif dan verifikasi inspeksi kontainer melalui perintah `docker inspect` ditunjukkan pada Gambar 8.

```
        max-size: "10m"
        max-file: "3"

--
      start_period: 15s
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"

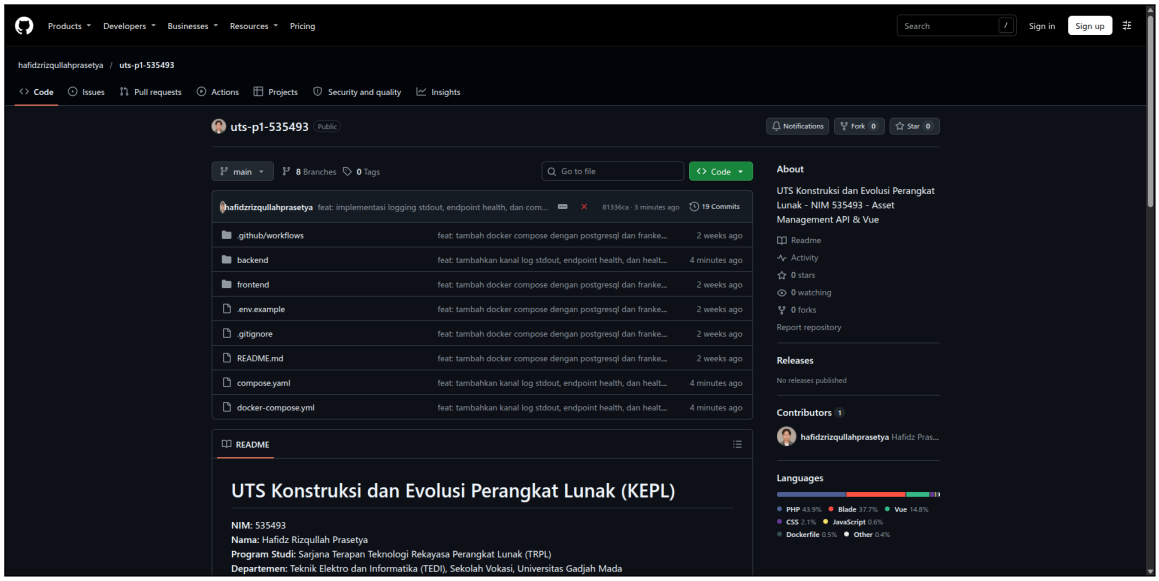
--
      - "${FRONTEND_PORT:-8080}:80"
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"

--- Inspeksi Kontainer ---
Logging Driver: json-file, Options: {"max-file":"3","max-size":"10m"}
```

Gambar 8: Konfigurasi `max-size` dan `max-file` serta hasil inspeksi runtime kontainer.

3.7 Penyelarasan Repositori GitHub

Seluruh penambahan kode pengendali `HealthController`, pembaruan `AssetController`, pengujian otomatis `HealthCheckTest`, serta berkas `compose.yaml` telah digabungkan ke cabang utama (`main`) melalui Pull Request nomor 11. Bukti komit terbaru pada repositori publik GitHub ditampilkan pada Gambar 9.



Gambar 9: Tampilan repositori publik GitHub memuat pembaruan berkas tugas 6 pada cabang `main`.

Chapter 4

Kesimpulan

Berdasarkan seluruh rangkaian perancangan, implementasi kode, dan pengujian sistem yang telah diselesaikan pada penugasan ini, diperoleh kesimpulan sebagai berikut:

1. Konfigurasi kanal pencatatan log pada Laravel berhasil dialihkan ke `stdout` melalui pengaturan `StreamHandler` pada berkas `logging.php` dan variabel lingkungan `LOG_CHANNEL=stdout`. Seluruh keluaran log aplikasi berhasil ditangkap langsung oleh perintah `docker compose logs` tanpa membebani media penyimpanan internal kontainer.

2. Tiga tingkatan log terstruktur (`info`, `warning`, dan `error`) telah diimplementasikan pada alur penyimpanan data aset, penolakan validasi formulir, dan kegagalan koneksi basis data. Masing-masing entri log membawa konteks pelaku, aktivitas, dan alasan secara terinci dengan tetap menjamin perlindungan privasi tanpa mencatat data kredensial maupun informasi sensitif.

3. Endpoint `/health` telah berhasil dibangun dengan memverifikasi kesiapan operasional basis data PostgreSQL dan subsistem penyimpanan *cache*. Endpoint ini secara konsisten mengembalikan kode status HTTP 200 OK saat seluruh komponen sehat, serta mengembalikan kode HTTP 503 Service Unavailable dan merinci komponen yang bermasalah saat terjadi pemutusan koneksi basis data.

4. Konfigurasi deklaratif *healthcheck* pada `compose.yaml` dengan interval 10 detik, batas waktu 5 detik, 5 kali percobaan ulang, dan masa tenggang awal 15 detik terbukti andal memantau status operasional kontainer backend. Kolom status pada perintah `docker compose ps` secara akurat merefleksikan kondisi kesehatan internal aplikasi.

5. Mekanisme pemantauan kesehatan terbukti bekerja secara adaptif dalam dua arah. Saat layanan basis data dimatikan secara sengaja, kontainer backend bertransisi secara mandiri menjadi berstatus *unhealthy*. Begitu layanan basis data dihidupkan kembali, status kontainer backend otomatis kembali menjadi *healthy* tanpa perlu melakukan intervensi manual terhadap proses peladen web.

6. Penerapan opsi pembatasan rotasi log dengan driver `json-file`, `max-size: "10m"`, dan `max-file: "3"` berhasil diaktifkan pada lingkungan Docker Compose, menjamin stabilitas konsumsi ruang penyimpanan peladen dari ancaman akumulasi berkas log tak terbatas.

Bibliography

[1] A. Wiggins, "The Twelve-Factor App: XI. Logs - Treat logs as event streams," 2017. [Daring]. Tersedia: https://12factor.net/logs. [Diakses: 06-Nov-2026].

[2] Docker Inc., "Configure logging drivers - JSON File logging driver," Docker Documentation, 2026. [Daring]. Tersedia: https://docs.docker.com/engine/logging/drivers/json-file/. [Diakses: 06-Nov-2026].

[3] J. Selda, "Monolog - Logging for PHP 8," GitHub Repository, 2026. [Daring]. Tersedia: https://github.com/Seldaek/monolog. [Diakses: 06-Nov-2026].

[4] OWASP Foundation, "Logging Cheat Sheet - Sensitive Data Protection and Storage," OWASP Cheat Sheet Series, 2026. [Daring]. Tersedia: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html. [Diakses: 06-Nov-2026].

[5] Docker Inc., "Dockerfile reference: HEALTHCHECK instruction," Docker Documentation, 2026. [Daring]. Tersedia: https://docs.docker.com/reference/dockerfile/#healthcheck. [Diakses: 06-Nov-2026].

[6] Compose Open Source Community, "Compose file specification: healthcheck attribute," 2026. [Daring]. Tersedia: https://docs.docker.com/compose/compose-file/05-services/#healthcheck. [Diakses: 06-Nov-2026].